Especificación de Diseño y Verificación: Unidad MAC con Algoritmo de Booth (Radix-4)

Israel Frausto VIlla

Índice

1. Introducción y Motivación **3**
1.1. Roadmap de Desarrollo (Fases del Proyecto) 3

2. Especificaciones del Sistema **3**
2.1. Parámetros de Operación Base 3

3. Arquitectura General (Microarquitectura) **4**
3.1. Jerarquía de Diseño 4
3.2. Unidad de Control (FSM) 4
3.3. Unidad de Acumulación 5
3.4. Camino de Datos (Datapath y ALU) 5

4. Entorno de Verificación **5**
4.1. Arquitectura del Testbench 5
4.2. Plan de Pruebas Ejecutado 6
4.3. Aserciones (SVA) y Cobertura Funcional 6

5. Resultados y Conclusiones **7**
5.1. Análisis de Resultados 7
5.2. Conclusiones 7
5.3. Trabajo Futuro 7

A. Apéndices: Código Fuente **8**
A.1. Módulos de Hardware (RTL) 8
A.1.1. `mac_defs.svh` (Definiciones Globales) 8
A.1.2. `mac_top.sv` (Nivel Superior) 9
A.1.3. `booth_multiplier.sv` (Multiplicador de Booth) 10
A.1.4. `booth_fsm.sv` (Unidad de Control) 11
A.1.5. `booth_datapath.sv` (Camino de Datos) 12
A.1.6. `accumulator_unit.sv` (Unidad de Acumulación) 14
A.1.7. `mac_adder.sv` (Sumador MAC) 14
A.2. Entorno de Verificación 15
A.2.1. `mac_pkg.sv` (Paquete de Clases) 15
A.2.2. `mac_if.sv` (Interfaz del Sistema) 16
A.2.3. `mac_asserts.sv` (Aserciones SVA) 17

A.2.4. `mac_cov.sv` (Cobertura Funcional) 18
A.2.5. `mac_tb.sv` (Testbench Principal) 20

1. Introducción y Motivación

La unidad Multiply-Accumulate (MAC) es el corazón de los sistemas de Procesamiento Digital de Señales (DSP). Operaciones críticas como filtros FIR/IIR, Transformadas Rápidas de Fourier (FFT) y, más recientemente, el cálculo masivo de tensores en aceleradores de Inteligencia Artificial (IA), dependen fundamentalmente de la operación matemática:

$$S = S + (A \times B)$$

El objetivo principal de este proyecto es diseñar, implementar y verificar una unidad MAC eficiente en hardware, optimizada para manejar números con signo en formato de Complemento a 2 de manera nativa, eliminando por completo la necesidad de conversores externos.

Para alcanzar una alta eficiencia temporal y de área, el núcleo del multiplicador se basa en el **Algoritmo de Booth con codificación Radix-4** (Booth Modificado). A diferencia del multiplicador tradicional que requiere N ciclos para operandos de N bits, esta arquitectura analiza los bits del multiplicador en ventanas superpuestas de 3 bits, permitiendo procesar operaciones complejas $(+M, -M, +2M, -2M, 0)$ y realizar desplazamientos de 2 bits por ciclo. Esto reduce la latencia de la multiplicación a exactamente la mitad ($N/2$ ciclos), optimizando drásticamente el rendimiento del sistema.

1.1. Roadmap de Desarrollo (Fases del Proyecto)

El desarrollo de esta unidad MAC se estructuró en cuatro fases evolutivas, permitiendo un diseño modular y una verificación incremental robusta:

- **Fase 1 (Completado):** Implementación y verificación del multiplicador base utilizando el algoritmo de Booth y la máquina de estados finitos (FSM) de control.
- **Fase 2 (Completado):** Conversión a Unidad MAC mediante la integración de un sumador combinacional parametrizado y un registro de acumulación de alta capacidad (72 bits) para prevenir desbordamientos (*overflow*).
- **Fase 3 (Completado):** Evolución a Radix-4 para la reducción de latencia (implementada exitosamente en el *datapath* actual, logrando la meta de $N/2$ ciclos).
- **Fase 4 (No realizado / Planeado):** *Pipelining*. Romper el camino crítico insertando registros entre las etapas de lógica combinacional para aumentar la frecuencia máxima de reloj (F_{max}) y el *throughput* de datos.

2. Especificaciones del Sistema

El diseño de la unidad MAC fue concebido con una arquitectura altamente parametrizable, permitiendo su escalabilidad para diferentes aplicaciones de procesamiento sin necesidad de reescribir el código RTL.

2.1. Parámetros de Operación Base

La configuración global del sistema se centraliza en el paquete de definiciones `mac_defs.svh`, estableciendo las siguientes características para la implementación actual:

- **Ancho de Datos (`MAC_DATA_WIDTH`):** 32 bits para los operandos de entrada (multiplicando y multiplicador).
- **Ancho del Acumulador (`MAC_ACC_WIDTH`):** 72 bits. Este sobredimensionamiento previene el desbordamiento (*overflow*) aritmético durante la ejecución de ciclos extensos de acumulación masiva (típicos en filtros digitales).
- **Límites Matemáticos:** El sistema define automáticamente los valores `MAC_MAX_POS` y `MAC_MAX_NEG` en función del ancho de datos para facilitar la verificación de los casos de esquina (*corner cases*).
- **Representación Numérica:** Manejo nativo de números con signo bajo el estándar de Complemento a 2.

3. Arquitectura General (Microarquitectura)

El sistema MAC ha sido diseñado siguiendo una metodología modular, separando la lógica de control de las unidades de procesamiento aritmético. Esta estructura permite una mayor escalabilidad y facilita la verificación independiente de cada bloque.

3.1. Jerarquía de Diseño

La organización del proyecto se divide en tres niveles jerárquicos principales, como se detalla en la Tabla 1. El nivel superior (`mac_top`) actúa como el orquestador entre el núcleo multiplicador y la unidad de almacenamiento.

Nivel	Módulo	Función principal
Top	`mac_top`	Interconexión final entre el multiplicador de Booth y la unidad de acumulación.
Nivel 1	`booth_multiplier`	Encapsula la lógica de multiplicación, integrando la FSM y el Datapath.
Nivel 1	`accumulator_unit`	Gestiona el registro de alta precisión y la lógica de suma histórica.
Nivel 2	`booth_fsm`	Implementa la lógica de control para el algoritmo de Booth Radix-4.
Nivel 2	`booth_datapath`	Ejecuta las operaciones de suma, resta y desplazamiento de bits.
Nivel 2	`adder_40bit`	Sumador combinacional utilizado para la acumulación.

Tabla 1: Jerarquía detallada de los módulos RTL del sistema.

3.2. Unidad de Control (FSM)

La máquina de estados finitos (`booth_fsm`) es el cerebro del multiplicador. Su diseño optimizado para **Radix-4** permite que el ciclo de cálculo sea de exactamente $N/2$ ciclos de reloj. Los estados implementados son:

- **IDLE:** Estado inicial que espera la señal de `start`.
- **LOAD:** Ciclo único donde se cargan los operandos `m_in` y `q_in` en los registros internos.
- **CALC:** Estado de computación donde se ejecutan las iteraciones de Booth. El sistema permanece aquí hasta que el contador de bits llega a 15 (para datos de 32 bits).
- **DONE:** Genera el pulso de `ready`, indicando que el producto de 64 bits es válido.

3.3. Unidad de Acumulación

El módulo `accumulator_unit` es un bloque síncrono encargado de integrar los productos sucesivos. Su funcionamiento es disparado por la señal `ready` del multiplicador, que actúa como la habilitación `acc_en`.

El producto entrante sufre una **extensión de signo segura** antes de alimentar a un sumador combinacional que lo integra al valor previamente almacenado. El registro de acumulación posee un ancho sobredimensionado de **72 bits** para soportar el crecimiento de la palabra tras miles de sumas consecutivas, garantizando la integridad de los datos (sin *overflow*) en aplicaciones de procesamiento DSP.

3.4. Camino de Datos (Datapath y ALU)

El `booth_datapath` implementa la lógica combinacional para la selección de la operación aritmética basada en la ventana de 3 bits del multiplicador: $\{Q_1, Q_0, q_{prev}\}$.

- **Extensión de Signo:** Para evitar errores de desbordamiento durante operaciones críticas como $+2M$ o $-2M$, el diseño extiende internamente los operandos a 18 bits (para el núcleo base) mediante la duplicación del bit de signo: `{{2{A[MSB]}}, A}`.

- **Lógica Radix-4:** A diferencia del Booth estándar, este datapath permite realizar desplazamientos aritméticos a la derecha de 2 bits por ciclo (¿2), reduciendo drásticamente el tiempo de ejecución.

  Multiplexor de Operaciones: Basado en la ventana de bits, la ALU selecciona entre: No operación, $\pm M$ o $\pm 2M$ (implementado mediante un desplazamiento a la izquierda del multiplicando).

4. Entorno de Verificación

El entorno de verificación fue diseñado con una arquitectura modular, separando la generación de estímulos, la encapsulación de señales, la validación del comportamiento (*self-checking*) y la medición de cobertura.

4.1. Arquitectura del Testbench

El entorno se compone de los siguientes elementos en SystemVerilog:

- **Paquete OOP (`mac_pkg.sv`):** Define la jerarquía de clases para la generación de estímulos aleatorios. La clase base `random_gen` restringe los valores al rango físico de los bits, mientras que `random_gen_corners` utiliza el operador `dist` para inyectar con mayor probabilidad valores extremos (`MAX_POS`, `MAX_NEG`) y ceros lógicos.
- **Interfaz (`mac_if.sv`):** Encapsula las señales del DUT y proporciona tareas (*tasks*) automáticas como `initialize()` y `compute()` para sincronizar los estímulos con el reloj del sistema, limpiando el código del bloque principal.
- **Testbench Top (`mac_tb.sv`):** Coordina la ejecución de los escenarios. Emplea directivas del compilador (`#ifdef`) para activar casos de prueba específicos. Internamente, utiliza aserciones en línea (`assert() else $error`) que funcionan como un *Scoreboard*, comparando el resultado de hardware contra el cálculo matemático ideal ($A \times B$).

4.2. Plan de Pruebas Ejecutado

El sistema fue sometido a pruebas de estrés intensivo, ejecutando millones de transacciones agrupadas en escenarios específicos:

ID	Macro de Prueba	Descripción del Escenario
1	`MAC_BASIC_FLOW_TEST`	10^6 operaciones aleatorias en dos ráfagas masivas. Incluye test de interrupción .ºn-fly"(`MAC_RST_MID_OP_TEST`) verificando la recuperación de la FSM al estado IDLE.
2	`MAC_SIGNED_MIX_TEST`	10^6 iteraciones con signos aleatorios para validar la aritmética en Complemento a 2.
3	`MAC_ACC_LOOP_TEST`	Ciclo ininterrumpido de 10^6 acumulaciones para estresar la capacidad de los 72 bits del acumulador.
4	`MAC_ZERO_OPS_TEST`	Multiplicación por cero lógico. Verifica que el acumulador preserve su estado anterior.
5	`MAC_MAX_MIN_TEST`	Inyección determinista de *Corner Cases*: `Max`×`Max`, `Min`×`Min` y `Max`×`Min`.
7	`MAC_CORNERS_TEST`	5×10^5 ciclos de inyección aleatoria ponderada forzando la aparición continua de valores extremos.

Tabla 2: Plan de pruebas implementado en `mac_tb.sv`.

4.3. Aserciones (SVA) y Cobertura Funcional

Para asegurar la integridad temporal y cuantificar la eficacia de las pruebas, se instrumentaron los siguientes monitores (vinculados al DUT mediante la sentencia `bind`):

- **Aserciones SVA (`mac_asserts.sv`):**

- `p_start_drops_ready`: Garantiza mediante implicación no superpuesta (`|=>`) que, al recibir la señal de `start`, la bandera `ready` cae de manera determinista en el siguiente ciclo de reloj.
- `assert_ready_not_x`: Asegura que la señal de salida no caiga en estados metaestables post-reset (`!$isunknown`).

- **Cobertura Funcional (`mac_cov.sv`):** El grupo `cg_mac` define *coverpoints* para cuantificar la estimulación del DUT. Divide los rangos operativos de los operandos en *bins* estratégicos (`zero, pos_small, neg_large, max_pos`, etc.). Además, el *coverpoint* `cp_fsm` verifica de manera estricta que la máquina de estados haya visitado todos los nodos previstos (`IDLE, LOAD, CALC, DONE`).

5. Resultados y Conclusiones

5.1. Análisis de Resultados

La fase de verificación exhaustiva demostró la robustez de la arquitectura RTL propuesta. El sistema superó con éxito las pruebas de estrés continuo, destacando los siguientes resultados métricos:

- **Precisión Aritmética:** Se ejecutaron más de 3.5 millones de transacciones aleatorias ponderadas (*corner cases*, signos mixtos y ceros) sin un solo desajuste entre el *Scoreboard* matemático del *testbench* y la salida del hardware.
- **Inmunidad al Desbordamiento:** Durante el escenario `MAC_ACCUM_LOOP_TEST`, el acumulador parametrizado de 72 bits procesó 1,000,000 de multiplicaciones sucesivas (operandos de 32 bits) comprobando que no existe pérdida de datos ni corrupción del bit de signo.
- **Resiliencia del Protocolo:** La prueba de interrupción .ªl vuelo" (`MAC_RST_MID_OP_TEST`) verificó que el sistema es capaz de abortar una multiplicación en pleno estado `CALC` y restaurar la FSM a `IDLE` de manera segura en un máximo de dos ciclos de reloj.

5.2. Conclusiones

El diseño de la Unidad MAC parametrizada cumplió íntegramente con los objetivos establecidos para las Fases 1 y 2. Se comprobó que la separación arquitectónica entre el Multiplicador de Booth y la Unidad de Acumulación facilita una síntesis limpia y evita condiciones de carrera durante la actualización del registro.

El uso de la extensión de signo interna `{{2{A[MSB]}}, A}` en la ALU demostró ser el mecanismo ideal para manejar el Complemento a 2 de manera nativa, descartando por completo la necesidad de bloques conversores externos que consumirían área lógica adicional. Finalmente, la metodología de verificación basada en clases y aserciones SVA garantizó un alto nivel de confianza (*sign-off*) en el núcleo RTL.

5.3. Trabajo Futuro

Para convertir este núcleo de procesamiento en una IP (*Intellectual Property*) competitiva para aplicaciones de alta velocidad, se plantea como siguiente paso la implementación

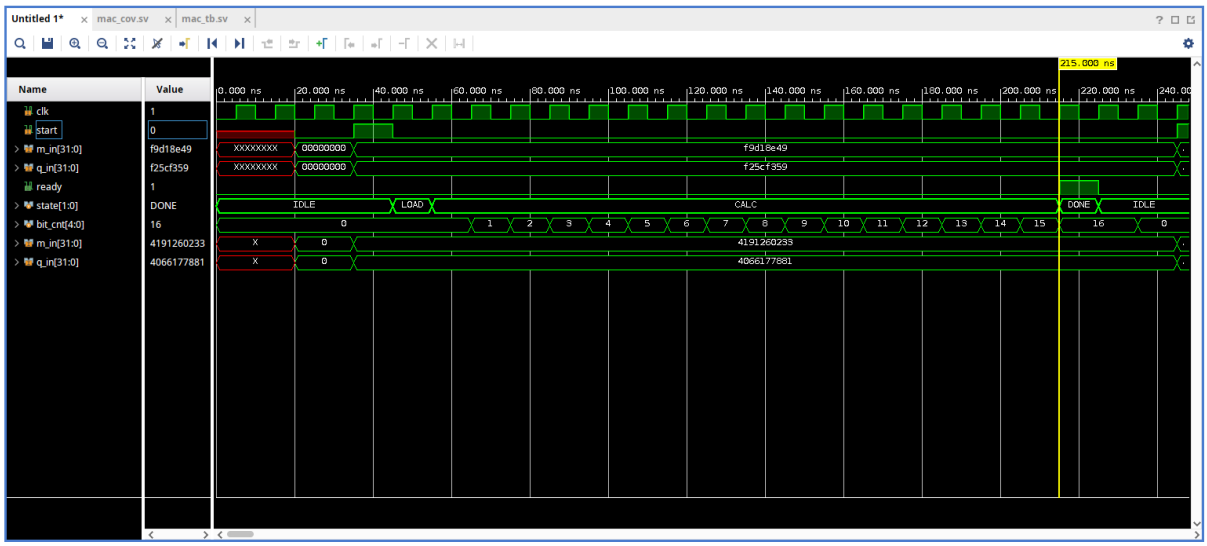

Figura 1: Enter Caption

de **Pipelining (Fase 4)**. Al segmentar el camino de datos e insertar registros entre el sumador de Booth y los registros de desplazamiento, se logrará romper el camino crítico, incrementando significativamente la frecuencia máxima de operación (F_{max}).

A. Apéndices: Código Fuente

En esta sección se adjunta el código fuente desarrollado en SystemVerilog, organizado de acuerdo con la jerarquía del diseño.

A.1. Módulos de Hardware (RTL)

A.1.1. `mac_defs.svh` (Definiciones Globales)

```
// mac_defs.svh
`ifndef MAC_DEFS_SVH
`define MAC_DEFS_SVH

// 1. Tama os base
`define MAC_DATA_WIDTH 32
`define MAC_ACC_WIDTH  72

// 2. L mites Matem ticos (Basados en DATA_WIDTH)
`define MAC_MAX_POS    ((1 << (`MAC_DATA_WIDTH-1)) - 1)
`define MAC_MAX_NEG    (-(1 << (`MAC_DATA_WIDTH-1)))

// 3. Rangos para Cobertura (Bins)
`define MAC_SMALL_POS_LIMIT  1000
`define MAC_SMALL_NEG_LIMIT -1000

`endif
```

Listing 1: Archivo de cabecera con parámetros escalables y límites matemáticos.

A.1.2. mac_top.sv (Nivel Superior)

```
/*************************************************************
 * NIVEL 1: TOP LEVEL - UNIDAD MAC COMPLETA
 * Este m dulo integra el Multiplicador de Booth y la Unidad de
    Acumulaci n.
 *************************************************************/
`timescale 1ns/1ps
module mac_top (
    input logic clk, rst_n, start,
    input logic signed [`MAC_DATA_WIDTH-1:0] m_in, q_in,
    output logic signed  [`MAC_ACC_WIDTH:0] product,
    output logic ready
);
    logic signed [`MAC_ACC_WIDTH-1:0] internal_product;
    logic mult_done;

    booth_multiplier multiplicador_inst (
        .clk(clk), .rst_n(rst_n), .start(start),
        .m_in(m_in), .q_in(q_in),
        .result(internal_product), .ready(mult_done)
    );

    accumulator_unit acumulador_inst (
        .clk(clk), .rst_n(rst_n), .acc_en(mult_done),
        .product_in(internal_product),
        .acc_out(product)
    );

    assign ready = mult_done;
endmodule
```

Listing 2: Módulo de integración principal (Top Level).

A.1.3. `booth_multiplier.sv` (Multiplicador de Booth)

```
`timescale 1ns/1ps

module booth_multiplier (
    input  logic clk, rst_n, start,
    input  logic [`MAC_DATA_WIDTH-1:0] m_in, q_in,
    output logic [`MAC_ACC_WIDTH-1:0] result,
    output logic ready
);
    // Se ales de interconexi n interna
    logic load, shift, op_sel;

    // Instancia de la FSM (Unidad de Control)
    booth_fsm control_unit (
        .clk(clk), .rst_n(rst_n), .start(start),
        .load(load), .shift(shift), .op_sel(op_sel), .ready(ready
           )
    );

    // Instancia del Datapath (Unidad Aritm tica)
    booth_datapath arithmetic_unit (
        .clk(clk), .rst_n(rst_n), .m_in(m_in), .q_in(q_in),
        .load(load), .shift(shift), .op_sel(op_sel), .product(
           result)
    );
endmodule
```

Listing 3: Módulo que encapsula la unidad de control y el camino de datos del multiplicador.

A.1.4. booth_fsm.sv (Unidad de Control)

```
`timescale 1ns/1ps

module booth_fsm (
    input  logic clk, rst_n, start,
    output logic load, shift, op_sel, ready
);
    typedef enum logic [1:0] {IDLE, LOAD, CALC, DONE} state_t;
    state_t state, next_state;
    logic [$clog2(`MAC_DATA_WIDTH)-1:0] bit_cnt;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state   <= IDLE;
            bit_cnt <= '0;
        end else begin
            state <= next_state;
            if (state == CALC) bit_cnt <= bit_cnt + 1'b1;
            else if (state == IDLE) bit_cnt <= '0;
        end
    end

    always_comb begin
        {load, shift, op_sel, ready} = '0;
        case (state)
            IDLE: next_state = start ? LOAD : IDLE;
            LOAD: begin
                load = 1'b1;
                next_state = CALC;
            end
            CALC: begin
                op_sel = 1'b1;
                shift  = 1'b1;
                // Condici n Radix-4: DATA_WIDTH / 2
                next_state = (bit_cnt == (`MAC_DATA_WIDTH/2)-1) ?
                    DONE : CALC;
            end
            DONE: begin
                ready = 1'b1;
                next_state = IDLE;
            end
            default: next_state = IDLE;
        endcase
    end
endmodule
```

Listing 4: Máquina de Estados Finitos (FSM) para el control del multiplicador Radix-4.

A.1.5. booth_datapath.sv (Camino de Datos)

```
`timescale 1ns/1ps

module booth_datapath (
    input  logic clk, rst_n,
    input  logic [`MAC_DATA_WIDTH-1:0] m_in, q_in,
    input  logic load, shift, op_sel,
    output logic [2*`MAC_ACC_WIDTH-1:0] product
);
    logic signed [`MAC_DATA_WIDTH-1:0] A, M;
    logic [`MAC_DATA_WIDTH-1:0] Q;
    logic q_prev;

    // ALU DE 18 BITS (Para evitar el desbordamiento del signo)
    logic signed [`MAC_DATA_WIDTH+1:0] ext_A;
    logic signed [`MAC_DATA_WIDTH+1:0] ext_M;
    logic signed [`MAC_DATA_WIDTH+1:0] alu_out;

    // Extendemos el signo copiando el bit m s significativo (MSB) DOS VECES
    assign ext_A = {{2{A[`MAC_DATA_WIDTH-1]}}, A};
    assign ext_M = {{2{M[`MAC_DATA_WIDTH-1]}}, M};

    // Multiplexor de suma/resta (Pura combinacional)
    always_comb begin
        case ({Q[1], Q[0], q_prev})
            3'b001, 3'b010: alu_out = ext_A + ext_M;
            3'b101, 3'b110: alu_out = ext_A - ext_M;
            3'b011:         alu_out = ext_A + (ext_M <<< 1);
            3'b100:         alu_out = ext_A - (ext_M <<< 1);
            default:        alu_out = ext_A;
        endcase
    end

    // SHIFT DE 34 BITS
    // Unimos los 18 bits de la ALU, los 16 de Q y 1 de q_prev = 35 bits
    logic signed [`MAC_ACC_WIDTH+2:0] shift_reg;
    assign shift_reg = {alu_out, Q, q_prev};

    // REGISTROS SECUENCIALES ---
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            {A, Q, q_prev} <= '0;
            M <= '0;
        end else if (load) begin
            A <= '0;
            M <= m_in;
            Q <= q_in;
            q_prev <= 1'b0;
```

```
        end else if (op_sel && shift) begin
            // Shift aritm tico seguro.
            // Tomamos los 33 bits inferiores [32:0] para
               guardarlos en {A, Q, q_prev}
            {A, Q, q_prev} <= (shift_reg >>> 2);
        end
    end

    assign product = {A, Q};

endmodule
```

Listing 5: Unidad aritmética para la selección de operaciones y desplazamientos.

A.1.6. accumulator_unit.sv (Unidad de Acumulación)

```
`timescale 1ns/1ps

module accumulator_unit (
    input  logic clk, rst_n, acc_en,
    input  logic signed [2*`MAC_DATA_WIDTH-1:0] product_in,
    output logic signed [`MAC_ACC_WIDTH:0] acc_out
);
    logic [`MAC_ACC_WIDTH:0] current_sum, acc_reg;
    logic [`MAC_ACC_WIDTH:0] product_ext;

    // Extensi n de signo segura (toma el bit MSB del producto)
    assign product_ext = product_in;

    // Instancia del sumador de 40 bits
    adder_40bit mac_adder (.a(product_ext), .b(acc_reg), .sum(
       current_sum));

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)          acc_reg <= '0;
        else if (acc_en)     acc_reg <= current_sum;
    end

    assign acc_out = acc_reg;
endmodule
```

Listing 6: Registro síncrono para la acumulación del producto.

A.1.7. mac_adder.sv (Sumador MAC)

```
`timescale 1ns/1ps

module adder_40bit (
    input  logic signed [`MAC_ACC_WIDTH:0] a,
    input  logic signed [`MAC_ACC_WIDTH:0] b,
    output logic signed [`MAC_ACC_WIDTH:0] sum
);
    assign sum = a + b;
endmodule
```

Listing 7: Sumador combinacional utilizado por el bloque acumulador.

A.2. Entorno de Verificación

A.2.1. mac_pkg.sv (Paquete de Clases)

```
package mac_pkg;
    // Clase Base: Generador Random
    class random_gen;
        rand bit signed [`MAC_DATA_WIDTH-1:0] a, b;
        constraint data_range {
            a inside {[`MAC_MAX_NEG : `MAC_MAX_POS]};
            b inside {[`MAC_MAX_NEG : `MAC_MAX_POS]};
        }
    endclass

    // Clase para Corners (Test 4, 5) Distribuci n Ponderada
    class random_gen_corners extends random_gen;
        constraint c_zeros {
           a dist {0:=20, `MAC_MAX_POS:=20, `MAC_MAX_NEG:=20,
              [1:`MAC_MAX_POS-1]:/20, [`MAC_MAX_NEG+1:-1]:/20};
            b dist {0:=20, `MAC_MAX_POS:=20, `MAC_MAX_NEG:=20,
               [1:`MAC_MAX_POS-1]:/20, [`MAC_MAX_NEG+1:-1]:/20};
        //el diagonal distribuye en partes iguales
        }
    endclass
endpackage
```

Listing 8: Clases base y extendidas para la generación aleatoria de estímulos y casos de esquina.

A.2.2. mac_if.sv (Interfaz del Sistema)

```
interface mac_if (input logic clk, input logic rst_n);
    logic start, ready;
    logic signed [`MAC_DATA_WIDTH-1:0] m_in, q_in;
    logic signed [`MAC_ACC_WIDTH-1:0] product;

    // Inicializaci n
    task automatic initialize();
        start <= 1'b0; m_in <= '0; q_in <= '0;
        @(posedge clk);
    endtask

    // Driver
    task automatic compute(input logic signed [`
       MAC_DATA_WIDTH-1:0] val_m, input logic signed [`
       MAC_DATA_WIDTH-1:0] val_q);
        @(posedge clk);
        start <= 1; m_in <= val_m; q_in <= val_q;
        @(posedge clk);
        start <= 0;
        wait(ready == 1'b1);
        repeat(2) @(posedge clk);
    endtask
endinterface
```

Listing 9: Encapsulamiento de señales y tareas (*tasks*) de inicialización y cómputo.

A.2.3. mac_asserts.sv (Aserciones SVA)

```
module mac_asserts (
    input logic clk, rst_n, start, ready,
    input logic signed [`MAC_DATA_WIDTH-1:0] m_in, q_in,
    input logic signed [`MAC_ACC_WIDTH-1:0] product
);

    // Regla: Si hay start, ready debe bajar al siguiente ciclo
    property p_start_drops_ready;
        @(posedge clk) disable iff (!rst_n) //si reset es 0, apaga assert
        start |=> !ready;   //Operador de implicaci n no superpuesta, si start es 1, baja ready al siguiente ciclo
    endproperty
    assert_start_drops_ready: assert property (p_start_drops_ready);

    // Regla: Ready no debe ser X despu s del reset
    assert_ready_not_x: assert property (@(posedge clk) rst_n |-> !$isunknown(ready));
    // Mientras reset enabled, rdy no ser  0

endmodule
```

Listing 10: Propiedades formales (*SystemVerilog Assertions*) para la validación del protocolo.

A.2.4. mac_cov.sv (Cobertura Funcional)

```
module mac_cov (
    input logic clk,
    input logic rst_n,
    input logic start,
    input logic signed [`MAC_DATA_WIDTH-1:0] operand_a,
    input logic signed [`MAC_DATA_WIDTH-1:0] operand_b,
    input logic [1:0]  state,
    input logic signed [`MAC_ACC_WIDTH-1:0] product_out,
    input logic ready
);

    // COVERGROUP PARA EL MAC
    covergroup cg_mac @(posedge clk iff rst_n);
        option.per_instance = 1;
        option.name = "Cobertura_Funcional_MAC";

        // Cobertura de Operandos (Signos y Casos Esquina) CORNERS DE PKG
        cp_a: coverpoint operand_a iff (state == 2'b01) {
            bins zero      = {0};
            bins pos_small = {[1 : `MAC_SMALL_POS_LIMIT]};
            bins pos_large = {[`MAC_SMALL_POS_LIMIT + 1 : `MAC_MAX_POS - 1]};
            bins neg_small = {[`MAC_SMALL_NEG_LIMIT : -1]};
            bins neg_large = {[`MAC_MAX_NEG + 1 : `MAC_SMALL_NEG_LIMIT - 1]};
            bins max_pos   = {`MAC_MAX_POS};
            bins max_neg   = {`MAC_MAX_NEG};
        }

        cp_b: coverpoint operand_b iff (state == 2'b01) {
            bins zero      = {0};
            bins pos_small = {[1 : `MAC_SMALL_POS_LIMIT]};
            bins pos_large = {[`MAC_SMALL_POS_LIMIT + 1 : `MAC_MAX_POS - 1]};
            bins neg_small = {[`MAC_SMALL_NEG_LIMIT : -1]};
            bins neg_large = {[`MAC_MAX_NEG + 1 : `MAC_SMALL_NEG_LIMIT - 1]};
            bins max_pos   = {`MAC_MAX_POS};
            bins max_neg   = {`MAC_MAX_NEG};
        }

        // Cobertura de la FSM (Booth)
        cp_fsm: coverpoint state {
            bins IDLE   = {2'b00};
            bins LOAD   = {2'b01};
            bins CALC   = {2'b10};
            bins DONE   = {2'b11};
            illegal_bins otros = default;
```

```
        }

        // COMBINACIONES (CROSS COVERAGE)
        a_x_b: cross cp_a, cp_b iff (state == 2'b01);

    endgroup

    // Instancia del grupo
    cg_mac cg_inst = new();

    // Reporte al final
    /*final begin
        $display("\n=== REPORTE DE COBERTURA FUNCIONAL (XCELIUM) ===");
        $display("Cobertura Operando A: %.1f%%", cg_inst.cp_a.get_coverage());
        $display("Cobertura FSM: %.1f%%", cg_inst.cp_fsm.get_coverage());
        $display("Cobertura Total: %.1f%%", cg_inst.get_coverage());
        $display("=============================================\n");
    end*/

endmodule

bind mac_top mac_cov u_mac_cov (
    .clk(clk),
    .rst_n(rst_n),
    .start(start),
    .operand_a(m_in),
    .operand_b(q_in),
    .product_out(product),
    .ready(ready),
    .state(multiplicador_inst.control_unit.state)
);
```

Listing 11: Definición de *covergroups* y *coverpoints* para cuantificar la estimulación del DUT.

A.2.5. mac_tb.sv (Testbench Principal)

```
`timescale 1ns/1ps

//`define MAC_BASIC_FLOW_TEST
// `define MAC_SIGNED_MIX_TEST
// `define MAC_ACCUM_LOOP_TEST
// `define MAC_ZERO_OPS_TEST
// `define MAC_MAX_MIN_TEST
// `define MAC_RST_MID_OP_TEST
 `define MAC_CORNERS_TEST

`include "mac_pkg.sv"
//`include "mac_if.sv"
//`include "mac_asserts.sv"

import mac_pkg::*;

module mac_tb;
    logic clk;
    logic rst_n;

    // Generador de Reloj Global
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Instancia de Interfaz y DUT
    mac_if if_i(clk, rst_n);
    mac_top dut (
        .clk(if_i.clk), .rst_n(if_i.rst_n),
        .start(if_i.start), .m_in(if_i.m_in), .q_in(if_i.q_in),
        .ready(if_i.ready), .product(if_i.product)
    );

    // Conexi n de Aserciones
    bind mac_top mac_asserts asserts_inst (
        .clk(clk), .rst_n(rst_n), .start(start),
        .ready(ready), .m_in(m_in), .q_in(q_in), .product(product
           )
    );

    // TEST CASE 1: BASIC FLOW
`ifdef MAC_BASIC_FLOW_TEST
    initial begin
    static logic signed [`MAC_DATA_WIDTH-1:0] rand_a, rand_b;
    static logic signed [`MAC_ACC_WIDTH-1:0] expected_acc = 0;

        $display("==================================");
        $display("TEST CASE 1: RANDOM FLOW");
```

```
        $display("==================================");

        rst_n = 0; #20ns; rst_n = 1;
        if_i.initialize();

        // Primera r faga (500mil c lculos)
        $display("Iniciando primera r faga de 500mil acumulaciones...");
        repeat(500000) begin
            rand_a = $random; rand_b = $random;
            expected_acc = expected_acc + ($signed(rand_a) * $signed(rand_b));
            if_i.compute(rand_a, rand_b);
            assert(if_i.product === expected_acc) else $error("Error en acumulaci n");
        end

        // Reset intermedio (Para limpiar y probar la FSM)
        $display("Aplicando reset manual...");
        rst_n = 0; #20ns; rst_n = 1;
        if_i.initialize();
        expected_acc = 0;

        // Segunda r faga (otros 500mil calculos)
        $display("Segunda r faga tras reset");
        repeat(500000) begin
            rand_a = $random; rand_b = $random;
            expected_acc = expected_acc + ($signed(rand_a) * $signed(rand_b));
            if_i.compute(rand_a, rand_b);
            assert(if_i.product === expected_acc) else $error("Error tras reset");
        end

        $display("TEST CASE 1 PASSED (Total: 1millon de pruebas)");

        // ============================================================
        // TEST CASE 6: RESET ON-FLY (Dentro del bloque del Test 1)
        // ============================================================
        $display("==========================================");
        $display("RUNNING: TEST CASE 6 (Reset On-Fly)");
        $display("==========================================");

        if_i.start <= 1'b1;
        if_i.m_in <= 32'h1234;
```

```
        if_i.q_in <= 32'h5678;

        repeat(5) @(posedge clk);
        $display("@%0t ns: Interrumpiendo operaci n con RESET...", $time);

        rst_n = 1'b0;
        repeat(2) @(posedge clk);
        rst_n = 1'b1;

        @(posedge clk);
        if (if_i.ready == 0 && if_i.product == 0) begin
            $display("@%0t ns: SUCCESS - Recuperaci n exitosa.", $time);
            $display("TEST CASE 6 PASSED");
        end else begin
            $display("@%0t ns: ERROR - No se recuper  bien.", $time);
            $display("TEST CASE 6 FAILED");
        end
        $display("=============================================\n");

        $finish;
    end
`endif

    // TEST CASE 2: SIGNED MIX (Signos Aleatorios)
`ifdef MAC_SIGNED_MIX_TEST
    random_gen rg;
    logic signed [`MAC_ACC_WIDTH-1:0] tb_acc;
    logic signed [`MAC_ACC_WIDTH-1:0] mult_res;

    initial begin
        rg = new();
        $display("==================================");
        $display("TEST CASE 2: SIGNED MIX");
        $display("==================================");

        // Reset Limpio
        tb_acc = '0;
        rst_n = 0;
        #20ns;
        rst_n = 1;
        if_i.initialize();

        // 1 millon Pruebas
        repeat(1000000) begin
            void'(rg.randomize());

            // Scoreboard
```

```
            mult_res = $signed(rg.a) * $signed(rg.b);
            tb_acc += mult_res;

            // Enviar al Hardware
            if_i.compute(rg.a, rg.b);

            // El Juez
            assert(if_i.product === tb_acc) else
                $error("ERROR Entradas: A=%0d, B=%0d | Suma Esperada=%0d | DUT escupi =%0d",
                     $signed(rg.a), $signed(rg.b), tb_acc, $signed(if_i.product));
        end

        $display("TEST CASE 2 PASSED");
        $finish;
    end
`endif

    // TEST CASE 3: ACCUM LOOP (Acumulaci n larga)
`ifdef MAC_ACCUM_LOOP_TEST
    random_gen_small rg_small;
    logic signed [`MAC_ACC_WIDTH-1:0] tb_acc;
    logic signed [`MAC_ACC_WIDTH-1:0] mult_res;

    initial begin
        rg_small = new();
        $display("========================================");
        $display("TEST CASE 3: ACCUMULATION LOOP");
        $display("========================================");

        // 1. Reset Limpio
        tb_acc = '0;
        rst_n = 0;
        #20ns;
        rst_n = 1;
        if_i.initialize();

        // 1 millon Pruebas
        repeat(1000000) begin
            void'(rg_small.randomize());

            // Matem ticas nativas perfectas
            mult_res = $signed(rg_small.a) * $signed(rg_small.b);
            tb_acc += mult_res;
            if_i.compute(rg_small.a, rg_small.b);

            assert(if_i.product === tb_acc) else
                    $error("Fallo acumulador! A=%0d, B=%0d | DUT =%0d, Esperado=%0d",
```

```
                        $signed(rg_small.a), $signed(rg_small.b),
                            $signed(if_i.product), tb_acc);
        end

        $display("TEST CASE 3 PASSED: 1 millon ciclos acumulados
           con  xito .");
        $finish;
    end
`endif

    // TEST CASE 4: ZERO OPS (Multiplicaci n por Cero)
`ifdef MAC_ZERO_OPS_TEST
    initial begin
        $display("==================================");
        $display("TEST CASE 4: ZERO OPS");
        $display("==================================");

        rst_n = 0;
        #20ns;
        rst_n = 1;
        if_i.initialize();

        // Cargar algo primero: 10*10 = 100
        $display("Cargando base: 10 * 10 = 100...");
        if_i.compute(32'd10, 32'd10);

        // Multiplicar por Cero (A=0) -> Debe mantenerse en 100
        $display("Acumulando: 0 * 55...");
        if_i.compute(32'd0, 32'd55);
        assert(if_i.product === 64'd100) else $error("Fallo Mult
           por Cero (A). DUT=%0d", if_i.product);

        // Multiplicar por Cero (B=0) -> Debe mantenerse en 100
        $display("Acumulando: 99 * 0...");
        if_i.compute(32'd99, 32'd0);
        assert(if_i.product === 64'd100) else $error("Fallo Mult
           por Cero (B). DUT=%0d", if_i.product);

        $display("\nTEST CASE 4 PASSED");
        $finish;
    end
`endif

    // TEST CASE 5: MAX/MIN (Corner Cases)
`ifdef MAC_MAX_MIN_TEST
    logic signed [`MAC_ACC_WIDTH-1:0] tb_acc;
    logic signed [`MAC_ACC_WIDTH-1:0] mult_res;

    initial begin
        $display("==================================");
        $display("TEST CASE 5: MAX / MIN");
```

```
        $display("==================================");

        tb_acc = '0;
        rst_n = 0; #20ns; rst_n = 1;
        if_i.initialize();

        // 1. Max * Max
        $display("Max * Max...");
        mult_res = $signed(`MAC_MAX_POS) * $signed(`MAC_MAX_POS);
        tb_acc += mult_res;
        if_i.compute(`MAC_MAX_POS, `MAC_MAX_POS);
        assert(if_i.product === tb_acc) else $error("Fallo Max*
           Max. DUT=%0d", if_i.product);

        // 2. Min * Min
        $display("Min * Min...");
        mult_res = $signed(`MAC_MAX_NEG) * $signed(`MAC_MAX_NEG);
        tb_acc += mult_res;
        if_i.compute(`MAC_MAX_NEG, `MAC_MAX_NEG);
        assert(if_i.product === tb_acc) else $error("Fallo Min*
           Min. DUT=%0d", if_i.product);

        // 3. Max * Min
        $display("Max * Min...");
        mult_res = $signed(`MAC_MAX_POS) * $signed(`MAC_MAX_NEG);
        tb_acc += mult_res;
        if_i.compute(`MAC_MAX_POS, `MAC_MAX_NEG);
        assert(if_i.product === tb_acc) else $error("Fallo Max*
           Min. DUT=%0d", if_i.product);

        $display("TEST CASE 5 PASSED");
        $finish;
    end
`endif

    // TEST CASE 7: CORNER CASES AUTOM TICOS
`ifdef MAC_CORNERS_TEST
    random_gen_corners rg_corners;
    logic signed [`MAC_ACC_WIDTH-1:0] tb_acc;
    logic signed [`MAC_ACC_WIDTH-1:0] mult_res;

    initial begin
        rg_corners = new();
        $display("========================================");
        $display("TEST CASE 7: CORNER CASES");
        $display("========================================");

        // Reset Limpio
        tb_acc = '0;
        rst_n = 0; #20ns; rst_n = 1;
        if_i.initialize();
```

```

        $display("Dispara 500mil tiros");
        repeat(500000) begin
            void'(rg_corners.randomize());

            // Scoreboard Matem tico
            mult_res = $signed(rg_corners.a) * $signed(rg_corners.b);
            tb_acc += mult_res;

            // Enviar al Hardware
            if_i.compute(rg_corners.a, rg_corners.b);

            // El Juez
            assert(if_i.product === tb_acc) else
                $error("Fallo Corner Case: A=%0d, B=%0d | DUT=%0d | ESPERADO=%0d",
                    $signed(rg_corners.a), $signed(rg_corners.b), $signed(if_i.product), tb_acc);
        end

        $display("TEST CASE 7 PASSED");
        $finish;
    end
`endif

endmodule
```

Listing 12: Módulo superior de pruebas para la ejecución automatizada de múltiples escenarios.